DATA ENGINEERING * HEALTHCARE ANALYTICS

Clinical EHR Data Cleaning & Cascading Imputation Pipeline

Transforming 1,000 messy Electronic Health Records into an audit-compliant, zero-null clinical dataset in Python & Pandas.

98.3%

COHORT PRESERVED

17 ghost records pruned; 983 real patients kept

100%

NULL RESOLUTION

1,325 missing fields fully cleaned & imputed

HIPAA

AUDIT COMPLIANT

Explicit Yes/No flags for full data provenance

Daniel .A. Dongo

Data Analyst | Python * Pandas * Healthcare Analytics

SWIPE TO EXPLORE ->

The Challenge: Dirty Healthcare Data

Why generic data cleaning breaks downstream medical machine learning.

1. Composite Vitals

Vitals

* Blood pressure stored as text string: e.g., '120/80', preventing analysis.
* Whitespace and corrupted entries.
* Required systolic/diastolic splitting.

2. Corrupted Data Types

Types

* Age column contained verbal words: e.g., text 'forty' mixed with numbers.
* Decimal floats on completed years.
* Required regex mapping & Int64 casting.

3. Polymorphic Dates

Dates

* Visit dates recorded with mixed syntax: '2023_04_15' vs '15-Apr-2023 10:30'.
* Underscores and irregular delimiters.
* Solved via fuzzy token-based parsing.

4. The 'Ghost Patient' Risk

Critical

* Blindly imputing missing values fabricates artificial patient profiles.
* Naive deletion drops 10% of real cohort.
* Needed an objective clinical triage rule.

CORE CLINICAL GOAL:

How do we resolve 1,325 missing values across 1,000 records without inventing synthetic patients or destroying statistical power?

Solution: Clinical triage filtering + stratified cascading median imputation.

Pipeline Architecture & Workflow

A 5-stage production framework engineered for clinical data integrity.

01 Text Normalization & Vital Splitting

Decomposed 'blood_pressure' string into numeric 'systolic_bp' and 'diastolic_bp'.

Normalized verbal text ('forty' -> 40), stripped whitespace, standardized snake_case.

02 Resilient Date Normalization

Parsed polymorphic visit dates with dateutil.parser (fuzzy=True) to clean delimiters.

Converted 100% of dates to datetime64[ns] without losing records.

03 The 'Ghost Patient' Triage Filter

Audited 4 critical markers: age, blood_pressure, condition, cholesterol.

Pruned exactly 17 records (1.70%) missing >=3 markers, preserving 98.3% of cohort.

04 Hierarchical Cascading Median Imputation

Tier 1: (Age Bracket + Gender) Median -> Tier 2: Gender Median -> Tier 3: Global.

Preserves biological reality without skewing age-specific clinical distributions.

05 Regulatory Audit Flags & Outlier Review

Created boolean provenance flags (*_imputed) mapped to clean 'Yes' / 'No'.

Audited clinical physiological limits + statistical 3.0x IQR boundaries.

Code Snapshot: Decomposition & Triage

Vectorized Pandas operations for feature extraction and row filtering.

1. Vectorized Blood Pressure Splitting (Cell 21-24)

```python
# Split composite slash-delimited blood pressure string
df[['systolic_bp', 'diastolic_bp']] = df['blood_pressure'].str.split('/', expand=True)

# Strip whitespace and coerce to numeric
df['systolic_bp'] = pd.to_numeric(df['systolic_bp'].str.strip(), errors='coerce')
df['diastolic_bp'] = pd.to_numeric(df['diastolic_bp'].str.strip(), errors='coerce')

# Cast to nullable integer to maintain clean schema
df[['systolic_bp', 'diastolic_bp']] = df[['systolic_bp', 'diastolic_bp']].round().astype('Int64')
```

2. Clinical Triage Filter (Cell 27-32)

```python
# Define core clinical markers
critical_columns = ['age', 'blood_pressure', 'condition', 'cholesterol']

# Count missing clinical variables per patient row
missing_per_row = df[critical_columns].isna().sum(axis=1)

# Prune patients missing >=3 core clinical markers (17 rows / 1.70%)
# Prevents 'inventing patients' via excessive imputation
df = df[missing_per_row < 3].copy()
```

Code Snapshot: Cascading Imputation

Stratified median hierarchy preserving demographic variation.

3. Cascading Median Imputation & Audit Flags (Cell 39-50)

```
# Core vitals and laboratory readings requiring imputation
imputed_cols = ['systolic_bp', 'diastolic_bp', 'cholesterol']

for col in imputed_cols:
    df[col] = pd.to_numeric(df[col], errors='coerce')
    # 1. Stratified subgroup median (Age Bracket + Gender)
    subgroup_med = df.groupby(['age_group', 'gender'], observed=False)[col].transform('median')
    # 2. Gender fallback median
    gender_med   = df.groupby('gender', observed=False)[col].transform('median')
    # 3. Global population median
    global_med   = df[col].median()
    # 4. Apply hierarchical cascade fallback
    df[col] = df[col].fillna(subgroup_med).fillna(gender_med).fillna(global_med)

# Cast to Int64 and update regulatory audit flags (*_imputed) to Yes/No
df[imputed_cols] = df[imputed_cols].round().astype('Int64')
df[bool_cols] = df[bool_cols].replace({True: 'Yes', False: 'No'})
```

WHY HIERARCHICAL MEDIANS OVER GLOBAL MEANS?

A 25-year-old female and a 75-year-old male have completely different healthy vital ranges. **Stratifying by age bracket and biological sex preserves real clinical distributions.**

Project Results: Before vs. After

100% missing data resolution with full regulatory auditability.

1,000 -> 983

PATIENTS RETAINED

98.3% cohort preserved;
17 ghost profiles dropped.

1,325 -> 0

MISSING VALUES

100% missing data resolved
with zero nulls remaining.

10 -> 15

ATTRIBUTES

+5 engineered columns:
vitals, age brackets, audit flags.

100% AUDITABLE

REGULATORY PROVENANCE

Full transparency: all imputed
cells tracked with Yes/No flags.

EXPORTED DELIVERABLE: cleaned_healthcare_data.csv

* 983 analytics-ready records prepared for clinical ML and statistical models.

* Numeric systolic & diastolic vitals enabled for cardiovascular risk scoring.

* 100% auditable lineage adhering to HIPAA and FDA 21 CFR Part 11 standards.

Key Takeaways & GitHub Repository

Core engineering lessons and link to the open-source repository.

1. Domain Knowledge Trumps Generic Cleaning

Takeaway 1

* Healthcare data cannot be treated like generic e-commerce logs.
* Imputing everything blindly invents patients; listwise deletion destroys real patients.
* Triage filters based on core vitals strike the optimal statistical balance.

2. Explainability Matters in Regulated Environments

Takeaway 2

* Stratified cascading medians are fast, 100% explainable, and zero-overhead.
* Explicit boolean audit flags (*_imputed) ensure compliance with HIPAA and FDA 21 CFR.
* Critical for medical AI models where training data provenance is scrutinized.

EXPLORE THE FULL PROJECT ON GITHUB:

https://github.com/ELSHINE123/healthcare-data-cleaning-pipeline

Developed by: Daniel .A. Dongo

LinkedIn: linkedin.com/in/daniel-dongo

VIEW REPO *